

ABANGCEBU AI — SPRINT 1 CORE INFRASTRUCTURE

User Logout & Session Invalidation Specification

Server-Side Session Revocation, HTTP-Only Cookie Chunk Purging, Multi-Tab Coordination & Router Cache Invalidation

Jira Reference: SCRUM-58

Framework: Next.js 16 (App Router) + Supabase Auth (@supabase/ssr)

Status: APPROVED / DONE

Author: junrilldisoy90 (Engineering Team)

Reviewed By: Hermar Centillas (Lead / Scrum Master)

Sprint: Sprint 1 (Architecture & Governance)

GitHub Repository Source: [github.com/hermarDev/Abang\\_Cebu\\_AI\\_Group2/blob/main/docs/auth-logout-spec.md](https://github.com/hermarDev/Abang_Cebu_AI_Group2/blob/main/docs/auth-logout-spec.md)

1. Purpose & Architectural Objectives

In Metro Cebu's rental ecosystem, students from universities (CIT-U, USC, UC, UP Cebu), BPO professionals at Cebu IT Park, and local property owners frequently access AbangCebu AI from shared terminals, office desktops, and mobile devices over public Wi-Fi. A lingering session exposes sensitive Philippine phone numbers, unverified inquiry drafts, and verified landlord publishing privileges.

Logout must not be a superficial client-side redirect. It is an **atomic, zero-trust session termination pipeline** that revokes server refresh tokens, zeroes all chunked HTTP-only cookies, coordinates cross-tab teardown via HTML5 `BroadcastChannel`, purges Next.js Router Cache, and validates destination routes against open-redirect attacks.

**Zero-Trust Revocation Principle:** Once logout is confirmed, both the server-side refresh token family and all client HTTP-only cookies are immediately destroyed. If network connectivity fails during sign-out, optimistic local cookie zeroing ensures the user is never stranded in an authenticated state.

2. Logout & Invalidation State Machine

| Current State          | Trigger / Event                                                | Action Taken                                                                           | Next State             |
|------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------|
| ACTIVE_SESSION         | User clicks Sign Out or session idle timeout expires           | Collect <code>LogoutPayload</code> (scope, redirectUrl)                                | INITIATING_LOGOUT      |
| INITIATING_LOGOUT      | Dispatch Server Action or POST /api/auth/logout                | Transmit JWT bearer or session cookie to Next.js server                                | SERVER_REVOCATION      |
| SERVER_REVOCATION      | Server executes <code>supabase.auth.signOut({ scope })</code>  | Revoke session row in <code>auth.sessions</code> & refresh token family                | COOKIE_PURGING         |
| COOKIE_PURGING         | Server iterates cookie jar for <code>sb-* -auth-token.*</code> | Emit <code>Set-Cookie</code> headers with <code>Max-Age=0</code> & past epoch          | CLIENT_SYNCHRONIZATION |
| CLIENT_SYNCHRONIZATION | Client receives response; broadcasts to tabs                   | Dispatch <code>BroadcastChannel.postMessage('SIGNED_OUT');</code> ; flush router cache | POST_LOGOUT_ROUTING    |
| POST_LOGOUT_ROUTING    | Validate destination via <code>isSafeRedirectUrl</code>        | Navigate client to sanitized route (default: <code>/login?message=logged_out</code> )  | UNAUTHENTICATED        |

3. Revocation Scopes & Supabase Auth Mechanics

AbangCebu AI implements three granular invalidation scopes supported by Supabase GoTrue Auth:

- `scope: 'local'` (Default): Invalidates only the refresh token and session row corresponding to the current browser/device. Active sessions on other devices (e.g., user's mobile phone) remain uninterrupted.
- `scope: 'global'`: Purges all active sessions and refresh token families for the user across every device. Mandated upon password reset, account suspension, or when user selects "Log out of all devices".
- `scope: 'others'`: Terminates all remote sessions while preserving the current active session. Accessed via Account Security Settings.

```
// Server Action Implementation (Next.js 16 App Router)
export async function signOutAction(payload?: LogoutPayload): Promise<LogoutSuccessResponse> {
  const cookieStore = await cookies();
  const scope: LogoutScope = payload?.scope || 'local';

  const supabase = createServerClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
    cookies: {
      getAll: () => cookieStore.getAll(),
      setAll: (cookiesToSet) => {
        cookiesToSet.forEach(({ name, value, options }) => cookieStore.set(name, value, options));
      },
    },
  });

  // 1. Revoke session on Supabase server
  await supabase.auth.signOut({ scope });

  // 2. Validate safe redirect URL
  const redirectUrl = isSafeRedirectUrl(payload?.redirectUrl)
    ? payload!.redirectUrl!
    : SESSION_CONSTANTS.DEFAULT_LOGOUT_REDIRECT;
```

```
return {  
  success: true,  
  message: 'User session successfully terminated.',  
  data: { scope, redirectUrl, invalidatedAt: new Date().toISOString() },  
};  
}
```

## 4. HTTP-Only Cookie Chunk Invalidation Protocol

Next.js 16 App Router stores session tokens in HTTP-only cookies via `@supabase/ssr`. When session payloads exceed 4096 bytes, tokens are chunked across multiple cookies (e.g., `sb-*-auth-token.0`, `sb-*-auth-token.1`). Deletion requires systematic zeroing of all base and numbered segments.

| Cookie Directive | Target Value                  | Security & Browser Engine Rationale                                         |
|------------------|-------------------------------|-----------------------------------------------------------------------------|
| Max-Age          | 0                             | Enforces immediate eviction across Chromium, Blink, and Gecko engines.      |
| Expires          | Thu, 01 Jan 1970 00:00:00 GMT | Mandatory past-epoch fallback for legacy WebKit and mobile Safari.          |
| Path             | /                             | Must match creation path exactly to prevent phantom cookie retention.       |
| HttpOnly         | true                          | Prevents client-side script interference during deletion.                   |
| Secure           | true (Production)             | Guarantees deletion headers are only processed over encrypted TLS channels. |
| SameSite         | lax                           | Maintains CSRF protection during top-level post-logout navigations.         |

## 5. Multi-Tab Invalidation via BroadcastChannel

To eliminate zombie sessions in background tabs (e.g., a map tab left open while logging out from account settings), the application coordinates cross-tab teardown via the HTML5 `BroadcastChannel` API on channel `supabase.auth.token`.

```
// Client Broadcast Synchronization Listener
export function setupAuthBroadcastSync(router: AppRouterInstance): () => void {
  if (typeof window === 'undefined' || !('BroadcastChannel' in window)) return () => {};

  const channel = new BroadcastChannel(SESSION_CONSTANTS.BROADCAST_CHANNEL_NAME);
  channel.onmessage = (event: MessageEvent<BroadcastAuthMessage>) => {
    if (event.data?.type === 'SIGNED_OUT') {
      // Invalidate Next.js client Router Cache and navigate to unauthenticated view
      router.refresh();
      router.replace(event.data.redirectUrl || SESSION_CONSTANTS.DEFAULT_LOGOUT_REDIRECT);
    }
  };

  return () => channel.close();
}
```

## 6. Next.js App Router Cache & Route Protection

The Next.js App Router maintains an in-memory client-side Router Cache storing prefetched Server Component payloads. If a user logs out and presses the browser's "Back" button, cached rental inquiries or landlord dashboard cards could be temporarily rendered.

- **Client-Side `router.refresh()`** : Executed immediately upon receiving sign-out confirmation, purging client-side RSC segment caches.
- **Zero-Cache Headers on Protected Layouts**: Protected routes (`/dashboard`, `/landlord/*`, `/saved`) enforce: `Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate`.
- **Edge Middleware Gatekeeper**: Middleware re-evaluates all incoming requests. Zeroed cookies result in immediate HTTP 307 redirects to `/login`.

## 7. Post-Logout Routing & Open-Redirect Firewall

To prevent malicious actors from engineering open-redirect phishing attacks (e.g., `/auth/logout?redirectUrl=https://scam-cebu.com`), all post-logout destinations are evaluated by `isSafeRedirectUrl()`:

| Input Target URL          | Firewall Rule            | Validation | Resolved Destination      |
|---------------------------|--------------------------|------------|---------------------------|
| /search?city=Cebu+City    | Valid relative root path | PASSED     | /search?city=Cebu+City    |
| https://phishing-site.com | External absolute URL    | BLOCKED    | /login?message=logged_out |
| //malicious-domain.com    | Protocol-relative URL    | BLOCKED    | /login?message=logged_out |
| /\attacker.com            | Backslash path injection | BLOCKED    | /login?message=logged_out |

## 8. Offline Resilience & Admin Forced Expulsion

- **Offline / Intermittent Connectivity**: If a user traveling through Cebu transit corridors attempts to sign out while offline, network timeouts to `POST /api/auth/logout` will not trap the user. Optimistic local cookie zeroing and memory flushing ensure credentials cannot be extracted from the device.
- **Immediate Admin Account Suspension**: When an abusive landlord is suspended (`profiles.is_suspended = true`), the admin trigger executes `supabase.auth.admin.signOut(user_id, 'global')`, terminating all sessions across the platform. Next.js Edge Middleware immediately expels the user upon the subsequent HTTP request.

## 9. Verification & Definition of Done (DoD)

**Audit Sign-Off:** All acceptance criteria from Jira SCRUM-58 have been fully satisfied. TypeScript contracts in `src/types/auth.ts` compile with 0 errors and 0 lint warnings. Zero premature React/JSX UI components were introduced.